LAB REPORT 04

Switch Statement, Break Statement & Conditional Operator in C++

Computer Programming · UET Nowshera · Electrical Engineering

1 Objective

The aim of this lab was to explore three additional decision-making tools in C++: the **switch** statement for multi-way branching, the **break** statement for controlling fall-through, and the **conditional (ternary) operator** as a compact alternative to simple if-else expressions. Key outcomes include:

- Writing a **switch** statement and understanding how case matching works
- Using **break** to prevent unintended fall-through between cases
- Understanding the **default** section as a catch-all branch
- Applying the ternary operator **? :** to replace simple if-else expressions
- Identifying why switch cannot use floating-point or relational expressions as case labels

2 Theoretical Background

The switch Statement

The **switch** statement evaluates an integer expression and branches directly to the matching **case** label. The expression must be an integer type (including **char**); floating-point values are not permitted. Each case label must carry a unique integer constant followed by a colon. If no case matches, the optional **default** section executes as a catch-all.

The break Statement

Without **break**, execution falls through into every subsequent case to the closing brace. Inserting **break** at the end of each case exits the switch immediately. Intentional fall-through is occasionally used to share code between consecutive cases but must be clearly documented to avoid confusion.

The Conditional (Ternary) Operator

The **? :** operator takes the form **condition ? expr_true : expr_false** and evaluates to one of two expressions based on the condition. It is a concise replacement for simple if-else statements, often used inline within assignments or output statements.

3 Software Used

IDE	DEV-C++ IDE
COMPILER	C++ Compiler (bundled with DEV-C++)

4 Experimental Tasks

Task 1 — Basic switch with break: Character Input

A character is read and a switch routes to the correct message. Each case ends with **break** so only one message prints. The **default** handles unexpected input.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main() {
    char choice;
    cout << "Enter A, B, or C: ";
    cin >> choice;
    switch (choice) {
        case 'A': cout << "You entered A.\n"; break;
        case 'B': cout << "You entered B.\n"; break;
        case 'C': cout << "You entered C.\n"; break;
        default:  cout << "You did not enter A, B, or C!\n";
    }
    return 0;
}
```

Task 2 — switch Without break: Fall-Through Demonstration

All **break** statements are removed. Entering 'A' now causes execution to fall through every case and default, printing all four messages — demonstrating why **break** is essential.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main() {
    char choice;
    cout << "Enter A, B, or C: ";
    cin >> choice;
    switch (choice) {
        case 'A': cout << "You entered A.\n";
        case 'B': cout << "You entered B.\n";
        case 'C': cout << "You entered C.\n";
        default:  cout << "You did not enter A, B, or C!\n";
    }
    return 0;
}
```

Task 3 — Tracing a switch: Predicting Output

Here **funny = serious * 2 = 30** before the switch is reached. Case 30 matches, so **"That is serious."** prints and break exits the block.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main() {
    int funny = 7, serious = 15;
    funny = serious * 2;        // funny = 30
    switch (funny) {
        case  0: cout << "That is funny.\n";           break;
        case 30: cout << "That is serious.\n";         break;
        case 32: cout << "That is seriously funny.\n"; break;
        default: cout << funny << endl;
    }
    return 0;
}
```

Task 4 — Error Analysis: Invalid switch Usage with double

This program incorrectly uses a **double** variable and relational expressions as case labels — both are illegal. Case labels must be integer constants; the fix is to use an if-else ladder instead.

● ● ● C++ Code

```
// ERRORS identified:
// 1. switch expression is double (must be integer type)
// 2. case labels use relational expressions (must be constants)

// CORRECTED version using if-else:
if      (testScore < 60)  cout << "Grade: F\n";
else if (testScore < 70)  cout << "Grade: D\n";
else if (testScore < 80)  cout << "Grade: C\n";
else if (testScore < 90)  cout << "Grade: B\n";
else                      cout << "Grade: A\n";
```

Task 5 — Conditional Operator: Even / Odd Check

The ternary operator **? :** checks even/odd in a single inline expression passed directly to **cout**, replacing a full if-else block.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
    cout << ((num % 2 == 0) ? "Even\n" : "Odd\n");
    return 0;
}
```

Task 6 — Self-Test: Converting if-else to Conditional Expressions

Each if-else from the lecture self-test was rewritten as a single ternary expression.

● ● ● C++ Code

```
// A)  z = (x > y) ? 1 : 20;

// B)  population = (temp > 45) ? base * 10 : base * 2;

// C)  wages *= (hours > 40) ? 1.5 : 1;

// D)  cout << ((result >= 0) ? "The result is positive\n"
//                            : "The result is negative.\n");
```

5 Conclusion

This lab introduced three additional C++ constructs for decision-making. The **switch** statement provides a clean alternative to a long else-if ladder when branching on a single integer or character value. The critical role of **break** was demonstrated by comparing programs with and without it — omitting break causes unintended fall-through that executes every subsequent case. The error-analysis task reinforced that switch case labels must be integer constants; relational expressions and floating-point variables are not permitted.

The conditional operator offered a compact way to write simple if-else logic inline. Rewriting if-else statements as ternary expressions in the self-tests highlighted when this shorthand improves readability. Together, switch and the ternary operator expand the programmer's toolkit for writing concise, well-structured decision logic.